Lecture 26

Making sense of variants

What do you think is easier?

1. Find a million variants?

2. Explain what one variant does?

It is all about computing skill.

If you **don't know** how to automate the processes it then 1. is much harder (perhaps impossible). If you **do know** how to automate then 2. is harder.

Calling variants is "procedural"

Documented steps:

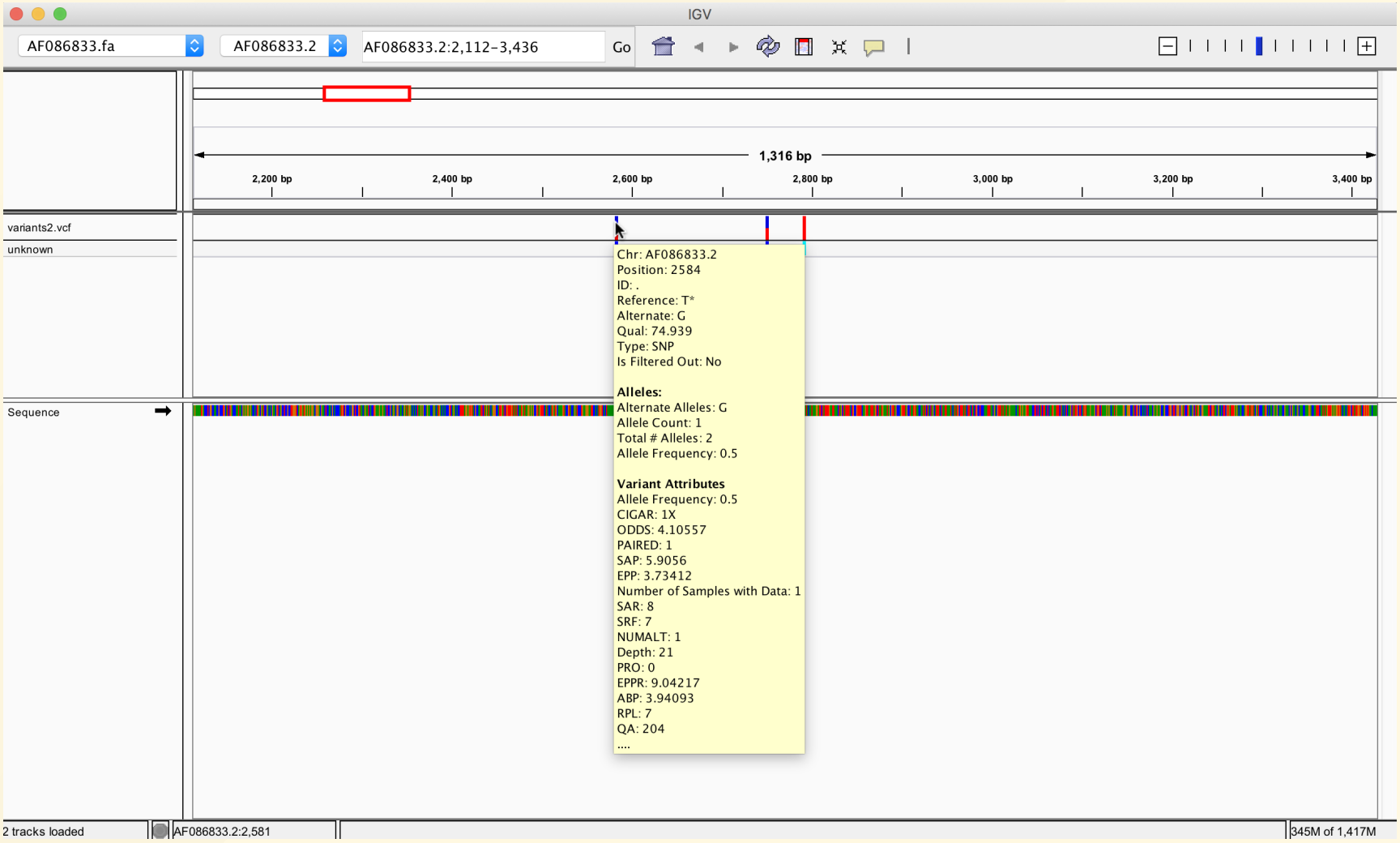
---

1. Align
2. Call variants
3. Filter variants

You still need to make decisions along the way.

Once you find variants, what do they mean?

Found the variant. Now what?



We need more "context".

The tortuous "history" of annotation representation.

You could also call it "sloppy" rather than "tortuous".

The problem appeared so simple "*it's just coordinates on a genome*" ... that no one really cared to properly define or enforce a format.

Not suprisingly the net effect is a "hodgepodge" of formats.

It is also typically unclear what each file may or may not contain. The ball is continusly punted to the *next generation*. Once it is untenable someone will fix it. Perhaps.

There are "formats" of course

What we call formats focus mostly on simple things: `start` should be in column `x` or `strand` is in column `y`.

But how do we represent relationships between concepts? And what is, or is not, included in a file?

Everyone is on their own there. You have to "know" how one specific group/database "does" it.

"Skill" is knowing where to look, whose data contains what you need/want 😕

Two major formats in use: BED vs GFF

- `BED` (Browser Extensible Data) originally specified by the UCSC genome browser
- `GFF` (General Feature Format) and has a bunch of variants: `GFF2`, `GTF` and `GFF3`

- `BED`:
  `[10, 15)` non-inclusive on the right: `10,11,12,13,14`.
  Indexing starts at `0`
- `GFF`:
  `[10, 15]` inclusive on both ends: `10,11,12,13,14,15`
  Indexing starts at `1`

BED vs GFF (part 2)

Obvious differences: the information is stored in different columns:

- `start` is column 2 in `BED`, column 3 in `GFF`
- May have columns not present in the other format.

Subtle but more important:

- `BED` format stores the information on the entire transcript on a single line
- `GFF` format stores the information on a transcript over multiple lines.

It is like Farenheit vs Celsius

- `BED` is "California Style" bioinformatics.
- `GFF` is "European Style" bioinformatics.

Where does this leave NCBI?

The NCBI policy used to be that they provide a GenBank file - you convert it to what you want.

Starting in 2016 it appears that that they have (tacitly) picked a side: `GFF`. NCBI data now can be obtained as `GFF`.

It is important to stick with one format.

Subtle errors may abound when converting between the two.

It is best to have all your data in one format.

Choose your interval format: BED or GFF

It feels a bit like these are your options:



Make a choice. Stick with it.

What should I use?

Wise man say: `GFF`

GFF will be discussed in more detail during in the RNA-Seq Lectures

Make a GFF file

Get a genbank file:

```
efetch -db nuccore -format gb -id AF086833 > AF086833.gb
```

Transform to GFF:

```
cat AF086833.gb | readseq -p -format=GFF | head
```

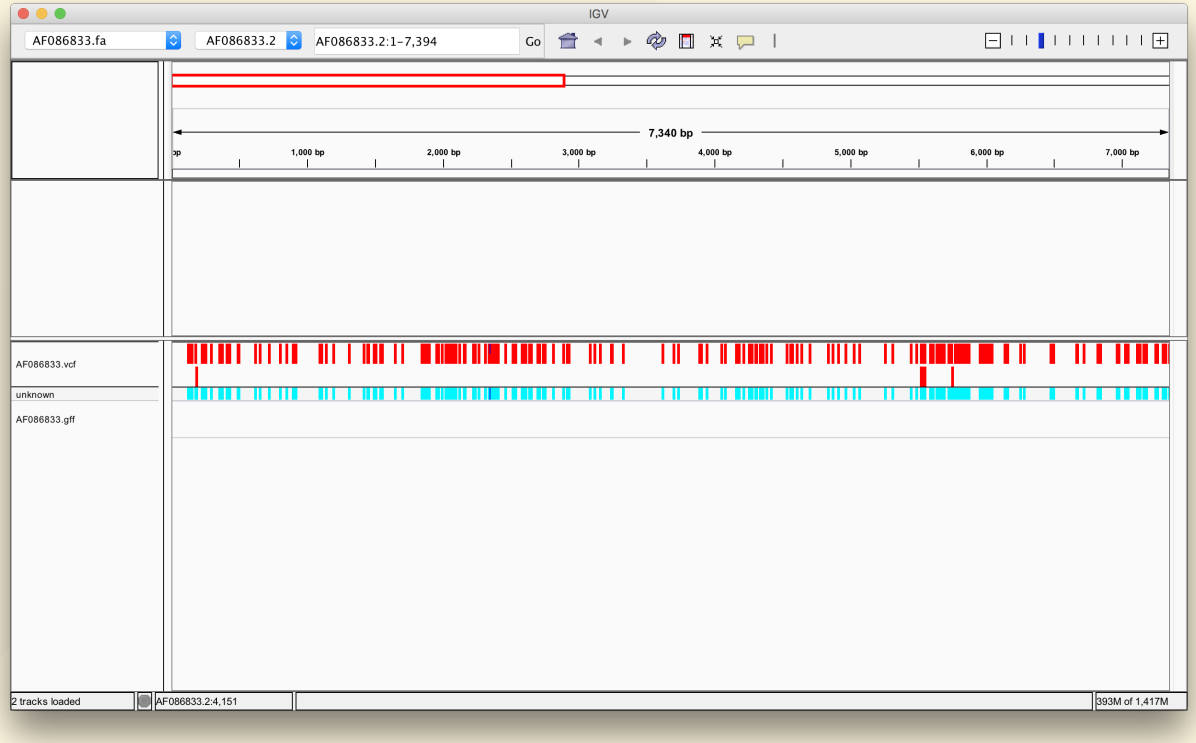
prints:

```
##gff-version 2
# seqname       source  feature start   end     score   st
AF086833        -       source  1       18959   .       +
AF086833        -       5'UTR   1       55      .       +
```

Visualize your GFF file

```
cat AF086833.gb | readseq -p -format=GFF > AF086833.gff
```

Load (drag) into IGV ... and ... the GFF track is empty!



Why is my IGV window empty?

This problem is common, ubiquitous and frustrating. The answer is almost always the same.

Your (sequence) chromosome names do not match the genome.

But, but ... I used the same data ....

Weel then one of the processes might changed the name of the sequence.

Here the version number gets dropped: `AF086833.2` becomes `AF086833`.

How do I fix this?

That's why you need a `megaton.sh` to fix one line and rerun the whole thing like nothing happened.

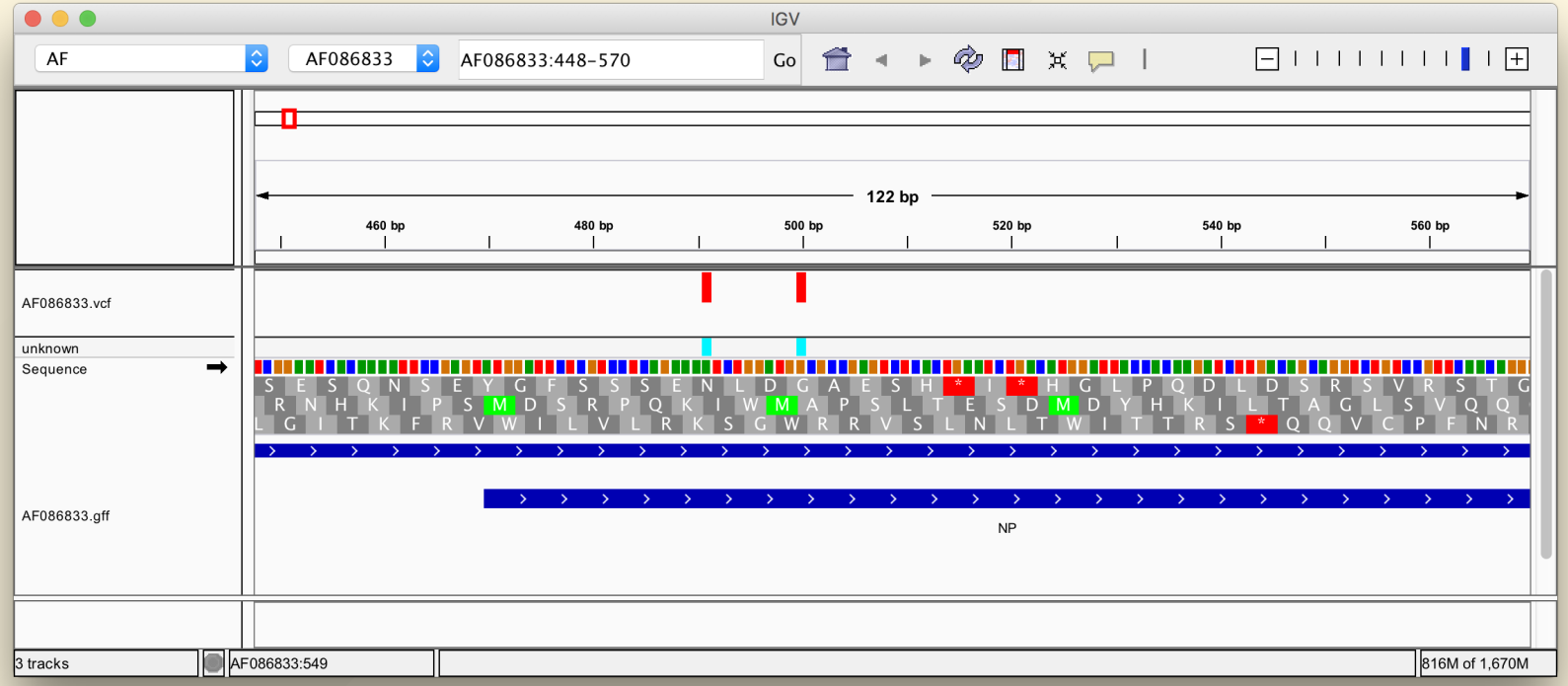
Convert the fasta file with the same tool

```
cat AF086833.gb | readseq -p -format=FASTA > AF086833.fa
```

Rerun your `megaton.sh`

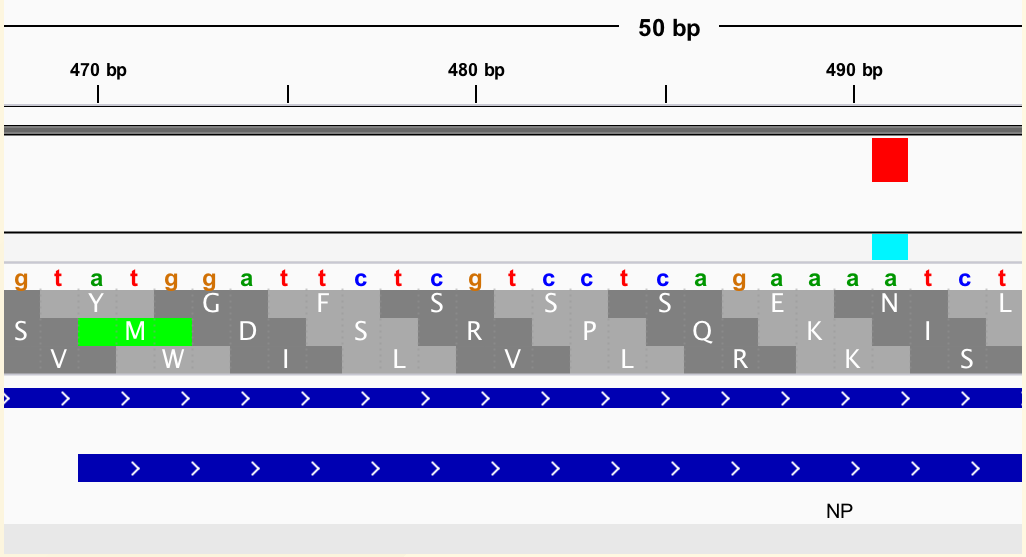
IGV visualization

Now drag the GFF file into IGV



Select "Show Translation" to see the amino acids.

You can visually "decode" the effect



Coding is in the second frame. Variant is homozygous `G`. This changes codon `ATC` (Isoleucine) to `GTC` (Valine). It is a non-synonymous substitution.

This is an "end-result" as far as sequence analysis goes - biological interpretion starts now.

Variant effect predictors

These tools do the same we did on the previous slide, and can automatically churn through a lot of data.

Running snpEFF

See the book chapter but in short you need a database to compare against:

```
snpEff download ebola_zaire
```

View the genome information:

```
snpEff dump ebola_zaire | head -3
```

Prints:

```
#-----------------------------------------
# Genome name                   : 'KJ660346.1'
# Genome version                : 'KJ660346.1'
```

The genome ID is different:

There are many ebola annotations snpEff uses `KJ660346`

```
Genome name                              : 'KJ660346.1'
```

Here is where having a `megaton.sh` script comes in handy, you can simply replace the accession number and just hit run.

(See the link to the script on the lecture page).

```
snpEff ebola_zaire KJ660346.vcf > annotated.vcf
```

The results of snpEff

A file called `snpEff_summary.html` is produced

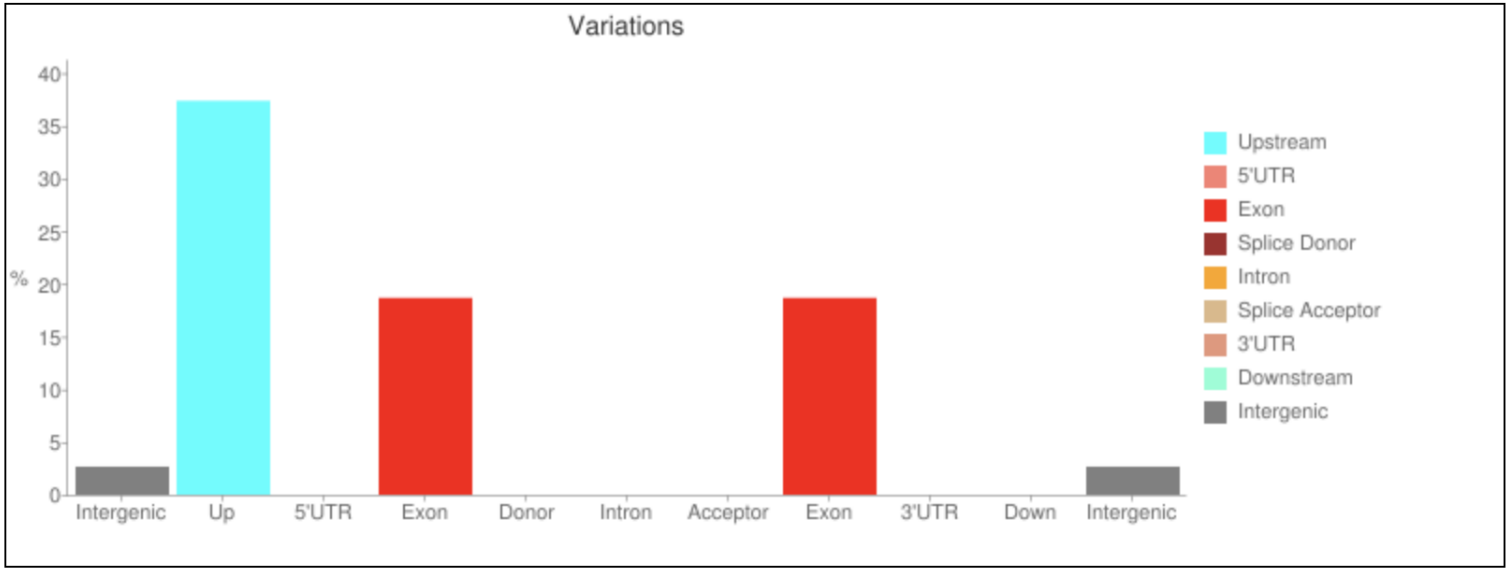
Number of effects by type and region

Type

Type (alphabetical order)	Count	Percent
downstream_gene_variant	31	41.333%
intergenic_region	2	2.667%
missense_variant	8	10.667%
synonymous_variant	6	8%
upstream_gene_variant	28	37.333%

Region

Type (alphabetical order)	Count	Percent
DOWNSTREAM	31	41.333%
EXON	14	18.667%
INTERGENIC	2	2.667%
UPSTREAM	28	37.333%



Variant effect predictor

Clinical interpretation is big business!

Even "free" tools may have a commercial component.

That alone is not the problem - it is tedious to keep them up to date.

See the book for alternatives.